# LeadForge

Assignment 1 — Lightweight Project Requirements Document

**Course:** Software Construction and Development

**Project:** LeadForge — Automated B2B Lead Generation, Enrichment, Outreach & CRM Pipeline

| Role | Member |
|---|---|
| Project Lead | Hikmatyar |
| Team Member | Muhannad Ayaz Khan |
| Team Member | Daniyal Hikmat |

## 1. Executive Overview

LeadForge is an end-to-end automated B2B lead generation, enrichment, cold outreach and CRM pipeline. It transforms raw business search queries into cleaned, enriched, scored and managed leads, then supports outreach, reply synchronization and CRM follow-up.

**Documented project version:** 2.0.0 Enterprise

**Documented project status:** Production Ready

The project documentation describes a FastAPI/Celery backend, a Next.js frontend, SQLite persistence, Redis background processing, Playwright scraping, Pandas data processing and OpenAI-assisted analysis/personalization.

## 2. Problem Statement

B2B prospecting normally requires repetitive work: finding businesses, collecting contact information, checking websites, identifying decision makers, evaluating opportunities, preparing personalized outreach, tracking replies and following prospects through a sales pipeline. LeadForge combines these activities into a single workflow to reduce manual effort and improve lead organization.

## 3. Objectives

• Automate business discovery from keyword, city and country queries.

• Clean, normalize and deduplicate lead records.

• Crawl websites and archive useful HTML pages for analysis.

• Extract emails, phone numbers, decision-maker information and social links.

• Audit website quality and identify sales opportunities.

• Generate deterministic lead scores and Hot/Warm/Cold tiers.

• Generate AI-assisted personalized outreach content.

• Manage multi-sender campaigns and follow-up sequences.

• Synchronize incoming email replies and update CRM stages.

• Provide CRM pipeline, tasks, activities and revenue tracking.

• Export lead data to CSV, Excel and Google Sheets.

## 4. Intended Users

Primary users are B2B sales, business-development and outreach operators who need to discover, qualify, contact and manage business prospects. Users can work with lead records, campaigns, inbox replies, CRM stages, tasks, exports and system settings.

## 5. Functional Requirements

| ID | Requirement |
|---|---|

| | |
|---|---|
| FR-01 | Accept manual and CSV/Excel business search queries using keyword, city and country. |
| FR-02 | Discover businesses using a Playwright-based Google Maps scraper. |
| FR-03 | Clean, normalize and deduplicate raw lead records. |
| FR-04 | Crawl selected business websites and cache raw HTML artifacts. |
| FR-05 | Extract emails, phone numbers, owner/decision-maker names and social links. |
| FR-06 | Perform website audits including SSL, CTA, contact-form, phone and responsive checks. |
| FR-07 | Use AI analysis to identify redesign/opportunity observations and outreach talking points. |
| FR-08 | Calculate deterministic 0–100 lead scores from multiple signals. |
| FR-09 | Assign Hot, Warm or Cold tiers from lead scores. |
| FR-10 | Create and manage campaigns, templates, senders and follow-up sequences. |
| FR-11 | Send configured SMTP outreach with daily limits, sender rotation and jitter controls. |
| FR-12 | Synchronize incoming IMAP replies and associate replies with leads. |
| FR-13 | Manage CRM stages from Prospect through Won/Lost with activities and follow-up tasks. |
| FR-14 | Provide dashboards, campaign statistics, pipeline and revenue views. |
| FR-15 | Export lead data to CSV, Excel and Google Sheets and retain export history. |

## 6. Non-Functional Requirements

- **Usability:** The UI should provide clear dashboards, tables, filters, Kanban pipeline views and reusable components.
- **Performance:** Long-running scraping/enrichment tasks should run asynchronously through Celery/Redis rather than blocking normal API requests.
- **Reliability:** Pipeline checkpoints, retries, job states and database-lock backoff should support recovery from interrupted processing.
- **Security:** API/service secrets must be stored through environment configuration and protected application settings; real secrets must never be committed to Git.
- **Maintainability:** Backend logic is separated into database, modules, routes, services, workers and utilities.
- **Quality:** The documented project uses pytest/pytest-cov and targets more than 80% backend coverage.
- **Scalability:** Configurable scraper concurrency, background workers and modular enrichment support larger workloads.

## 7. High-Level Architecture & Data Flow

| Stage | Main components | Output |
|---|---|---|
| 1. Ingestion | Manual keyword/city/country input; CSV/Excel query upload | Search queries |
| 2. Discovery | Playwright Google Maps scraper | Raw business leads |
| 3. Processing | Cleaner → crawler → email/phone/owner/social extraction → website audit | Enriched leads → AI analysis |
| 4. Scoring | Deterministic multi-signal scoring and tier engine | 0–100 score + Hot/Warm/Cold |
| 5. Outreach | Campaign engine → SMTP sender rotation → recipients | Outbound emails |
| 6. Inbox | IMAP synchronization + sentiment/stage processing | Replies linked to leads |
| **Stage** | **Main components** | **Output** |
| 7. Management | CRM Kanban → tasks → revenue; exports | Pipeline and business records |

## 8. Main Repository Structure

The source documentation identifies the project root and its main components. The repository includes Docker configuration, environment templates, backend code, frontend code, tests, utilities and generated/export data.

| Path | Purpose |
|---|---|
| backend/ | FastAPI application, database layer, modules, routes, services, utilities and Celery workers. |
| leadforge-ui/ | Next.js/React frontend, pages, components, API client and UI utilities. |
| tests/ | Automated API, scraper, cleaner, pipeline, scorer and export tests. |
| docs/ | Assignment requirements and supporting project documentation. |
| README.md | Project overview, setup, technology stack, status and contribution information. |
| requirements.txt | Python dependencies. |
| docker-compose.yml | Redis/backend/worker deployment configuration. |
| .env.example | Environment variable template without real secrets. |

## 9. Key Backend Modules

| Module | Purpose |
|---|---|
| maps_scraper.py | Playwright-based Google Maps discovery. |
| data_cleaner.py | Pandas-based normalization and deduplication. |
| pipeline.py | Master crash-resilient processing pipeline. |
| email_extractor.py / phone_extractor.py | Contact information extraction. |
| owner_finder.py / owner_detector.py | Decision-maker/owner detection. |
| social_scraper.py | Social profile discovery. |
| website_auditor.py | Website quality and opportunity analysis. |
| ai_analyzer.py | AI-based website/redesign analysis. |
| ai_personalizer.py | AI-assisted personalized outreach copy. |
| lead_scorer.py | Deterministic lead scoring and tiering. |
| export_manager.py | CSV, XLSX and Google Sheets exports. |

## 10. Database / Core Entities

The source documentation describes SQLAlchemy ORM models for the application's persistence layer, including lead records, website audits, campaigns, outreach emails and CRM records. The database uses SQLite with WAL mode in the documented local configuration.

| Entity | Representative data |
|---|---|
| Leads | Business identity, contact information, website, score, tier and enrichment fields. |
| Website Audits | Lead reference, audit score, platform, detected problems and opportunities. |
| Campaigns | Campaign name/status, daily limit, sent/opened/replied counters. |
| **Entity** | **Representative data** |
| Outreach Emails | Campaign, lead, recipient, subject, status and sent timestamp. |
| CRM Records | Lead, stage, deal value and priority. |

# 11. Prioritized Product Backlog

| Priority | Backlog item |
|---|---|
| P0 | Lead discovery, cleaning/deduplication, scoring, lead management and CRM pipeline. |
| P1 | Website crawling/auditing, AI analysis, personalization, campaigns, SMTP sending and IMAP synchronization. |
| P1 | CSV/Excel/Google Sheets export and operational settings. |
| P2 | Revenue analytics, forecasting and additional operational/health improvements. |

# 12. Assumptions, Constraints & Limitations

• Required external services and credentials must be available when their integrations are used.

• Redis is required for the documented Celery background-processing workflow.

• Chromium is required through Playwright for the scraper/crawler workflow.

• External websites, Google Maps pages, email providers and APIs may change availability or behavior.

• Scraped and AI-enriched information should be validated before being treated as authoritative.

• Real secrets, API keys and credentials must not be committed to the public repository.

• The documented test suite and CI configuration should be run in the actual submission environment to verify the currentbuild.

# 13. Deployment & Development Setup

## Documented local setup:

• Create a Python virtual environment and install requirements.txt.

• Install Chromium through Playwright.

• Create .env from .env.example and configure required services.

• Start Redis and the Celery worker.

• Start FastAPI on port 8000.

• Install frontend dependencies and run the Next.js frontend on port 3000.

The documented full-stack deployment also supports Docker Compose for Redis, the backend API and the Celery worker.

# 14. Testing & Quality Assurance

The project documentation specifies pytest and pytest-cov testing, including functional suites for API, scraper, cleaner, pipeline, scorer and export modules. The documented coverage command enforces an 80% minimum threshold, and GitHub Actions is configured to run tests on pushes and pull requests to main.

## Core commands:

pytest -v pytest --cov=backend --cov-report=term-missing --cov-fail-

under=80

# 15. Team Responsibilities

| Member | Initial responsibility |
|---|---|
| Hikmatyar — Project Lead | Project coordination, repository management, requirements/documentation, integr oversight. |
| Muhannad Ayaz Khan | Assigned development work across backend/frontend modules, testing and fixes. |
| Daniyal Hikmat | Assigned development work across backend/frontend modules, testing and fixes. |

## 16. Repository & Versioning Plan

The repository must remain public and maintain a clear history. Meaningful commits should be made by all three members using their own GitHub accounts. Use Git tags for significant versions instead of duplicate project copies. Suggested initial development tag: **v0.1.0**.